

Sau bài thực hành này, sinh viên có khả năng:

- Sử dụng thư viện Numpy để xử lý array và ma trận lớn

1. GIỚI THIỆU NUMPY

- Numpy là thư viện dành cho lập trình toán học, khoa học, kỹ thuật và khoa học dữ liệu.
- Numpy xử lý rất tốt các đối tượng mảng đa chiều và ma trận.
- Numpy xử lý khối lượng lớn dữ liệu rất nhanh
- Tensorflow và Scikit sử dụng Numpy để tính toán ma trận ở backend.

2. CÀI ĐẶT NUMPY

```
pip install numpy
```

3. TẠO NUMPY ARRAY

3.1. Tạo array 1-chiều

```
#cách 1
# tạo list
my_list = [1, 9, 15, 39]
print('Python list:', my_list)
#chuyển list --> Numpy array
my_numpy_array = np.array(my_list)
print('Numpy array:', my_numpy_array)

#cách 2
a = np.array([1, 9, 15, 39])
print(a)

Python list: [1, 9, 15, 39]
Numpy array: [ 1  9 15 39]
[ 1  9 15 39]
```

Ta có thể dùng thuộc tính shape, dtype để xem kích thước mảng, kiểu dữ liệu.

3.2. Array 2-chiều

```

▶ c = np.array([(1, 2, 3),
                (4, 5, 6)
                ])

print(c)
print(c.shape)
print(c.dtype)

```

```

[[1 2 3]
 [4 5 6]]
(2, 3)
int64

```

3.3. Array 3-chiều

```

▶ d = np.array([
                [[1., 2., 3.], [4., 5., 6.]],
                [[7., 8., 9.], [10, 11, 12]]
                ])

print(d)
print(d.shape)

```

```

[[[ 1.  2.  3.]
  [ 4.  5.  6.]]

 [[ 7.  8.  9.]
  [10. 11. 12.]]]
(2, 2, 3)

```

3.4. Tạo Array với giá trị ban đầu là 0 hoặc 1

```

▶ e = np.zeros((2, 2))
f = np.ones((3, 3))
print('Zero matrix:\n', e)
print('One matrix\n', f)

```

```

Zero matrix:
[[0. 0.]
 [0. 0.]]
One matrix
[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]

```

3.5. Reshape và Flatten ma trận

3.5.1. Reshape thay đổi kích thước ma trận

```

▶ e = np.array([[1., 2., 3.], [4., 5., 6.]])
print(e)
print(e.shape)
f = e.reshape(3,2)
print(f)
g = np.array([[1., 2., 3.], [4., 5., 6.]],
              [[7., 8., 9.], [10., 11., 12.]]
              )
print(g)
print(g.shape)
h = g.reshape(3,4)
print('reshape (3, 4)\n',h)
j = g.reshape(3, 2, 2)
print('reshape (3, 2, 2)\n', j)
k = g.reshape(2, 3, 2)
print('reshape (2, 3, 2)\n', k)

```

```

[[1. 2. 3.]
 [4. 5. 6.]]
(2, 3)
[[1. 2.]
 [3. 4.]
 [5. 6.]]
[[[ 1.  2.  3.]
 [ 4.  5.  6.]]

 [[ 7.  8.  9.]
 [10. 11. 12.]]]
(2, 2, 3)
reshape (3, 4)
[[ 1.  2.  3.  4.]
 [ 5.  6.  7.  8.]
 [ 9. 10. 11. 12.]]
reshape (3, 2, 2)
[[[ 1.  2.]
 [ 3.  4.]]

 [[ 5.  6.]
 [ 7.  8.]]

 [[ 9. 10.]
 [11. 12.]]]

```

3.5.2. Platten là chuyển array nhiều chiều về 1 chiều

```

▶ e = np.array([[1., 2., 3.], [4., 5., 6.]])
  f = e.flatten()
  print(f)
  g = np.array([[[1., 2., 3.], [4., 5., 6.]],
                [[7., 8., 9.], [10., 11., 12.]]
                ])
  j = g.flatten()
  print(j)

[1.  2.  3.  4.  5.  6.]
[ 1.   2.   3.   4.   5.   6.   7.   8.   9.  10.  11.  12.]

```

3.6. Thêm dòng dữ liệu với **hstack** và **vstack**

Hàm **hstack((x, y))** cho phép thêm nối thêm dữ liệu ma trận y vào sau ma trận x (chiều ngang).

Hàm **vstack((x, y))** cho phép thêm nối thêm dữ liệu ma trận y vào dưới ma trận x (chiều dọc).

```

▶ f = np.array([1., 2., 3.])
  g = np.array([4., 5., 6.])
  h_stack = np.hstack((f, g))
  print('Horizontal Append\n', h_stack)
  v_stack = np.vstack((f, g))
  print('Vertical Append\n', v_stack)

Horizontal Append
[1.  2.  3.  4.  5.  6.]
Vertical Append
[[1.  2.  3.]
 [4.  5.  6.]]

```

3.7. Sinh mảng số ngẫu nhiên

3.7.1. Sinh mảng số nguyên ngẫu nhiên

```

▶ a = np.random.randint(100, size=(5,))
  print('Random 1-D array\n',a)
  b = np.random.randint(100, size=(3, 4))
  print('Random 2-D array\n',b)
  c = np.random.randint(100, size=(3, 4, 3))
  print('Random 3-D array\n',c)

```

```

× Random 1-D array
  [ 2 75 62 94 66]
Random 2-D array
  [[98 92 94 97]
   [98 69 51 51]
   [50  9 48 84]]
Random 3-D array
  [[[58 40 14]
    [57 42 26]
    [16  9 41]
    [59 84 69]]

   [[62 23  7]
    [97  7 49]
    [36 80 51]
    [83 37 70]]

   [[ 5 86 12]
    [42  3 98]
    [79 23 12]
    [ 1 34 44]]]

```

3.7.2. Sinh mảng số thực trong khoảng [0, 1]

```
#sinh ngẫu nhiên mảng số thực [0, 1]
a = np.random.rand(5)
print('Random 1-D array\n',a)
b = np.random.rand(3, 4)
print('Random 2-D array\n',b)
c = np.random.rand(3, 4, 3)
print('Random 3-D array\n',c)
```

Random 1-D array
[0.97694795 0.58135609 0.37310049 0.04317114 0.95870681]

Random 2-D array
[[0.20065188 0.77415168 0.01271068 0.54287947]
[0.08131005 0.23991586 0.76875529 0.33362021]
[0.47339329 0.53452209 0.41521402 0.81569097]]

Random 3-D array
[[[0.45141181 0.80935643 0.9196085]
[0.17274109 0.05948666 0.01011627]
[0.08212584 0.88883691 0.30653759]
[0.46800835 0.24948332 0.36816523]]

[[0.31344616 0.52413104 0.87969277]
[0.80528164 0.06993265 0.67356058]
[0.82108591 0.58362639 0.01413421]
[0.69432265 0.67462178 0.89112465]]

[[0.96110129 0.2429993 0.2232738]
[0.69637372 0.72374122 0.63525499]
[0.06367548 0.87673229 0.18913645]
[0.2968745 0.50298948 0.15396021]]]

3.7.3. Sinh mảng số ngẫu nhiên theo phân phối chuẩn Gaussian

Ta dùng hàm

`Numpy.random.normal(loc, scale, size)`

Trong đó

Loc: giá trị trung tâm

Scale: độ lệch chuẩn

Size: số phần tử trả về

```
a = np.random.normal(5, 0.5, 10)
print(a)
```

[4.94284102 5.0238746 4.69654707 4.59024079 5.34002763 5.35437726
4.87984648 5.38341356 4.92519622 4.95704346]

3.7.4. Sinh ngẫu nhiên một giá trị từ mảng giá trị cho trước

```
#sinh 1 giá trị ngẫu nhiên từ mảng giá trị cho trước
a = np.random.choice([2, 4, 6, 8, 10, 12, 14, 16, 18, 20])
print(a)
#sinh ma trận 3 x 5 ngẫu nhiên từ mảng giá trị cho trước
b = np.random.choice([2, 4, 6, 8, 10, 12, 14, 16, 18, 20], size=(3, 5))
print(b)
```

```
4
[[ 2 20 18 14  4]
 [12 16  8  6 18]
 [ 2  2  2 16 14]]
```

3.8. Thay đổi giá trị của ma trận

Các giá trị trong ma trận là bất biến. Để thay đổi giá trị của ma trận ta làm như sau

```
#Tạo ma trận 5 x 5 với số 1
A = np.matrix(np.ones((5, 5)))
print('The matrix\n',A)
np.array(A)[2] = 2
print('Change values of third row with array\n', A)
np.asarray(A)[2] = 2
print('Change values of third row with asarray\n', A)
```

```
The matrix
[[1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]]
Change values of third row with array
[[1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]]
Change values of third row with asarray
[[1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]
 [2. 2. 2. 2. 2.]
 [1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]]
```

3.9. Sinh dãy số theo khoảng cho trước

3.9.1. Dùng hàm arange

```
#tạo 20 số nguyên từ 1 --> (n-1)
a = np.arange(1, 20)
print(a)
#tạo 20 số nguyên từ 1 --> (n-1), mỗi bước tăng 2 giá trị
b = np.arange(1, 20, 2)
print(b)
```

```
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19]
[ 1  3  5  7  9 11 13 15 17 19]
```

3.9.2. Dùng hàm linspace

```
a = np.linspace(1., 5., num=10)
print(a)
```

```
[1.          1.44444444 1.88888889 2.33333333 2.77777778 3.22222222
 3.66666667 4.11111111 4.55555556 5.          ]
```

3.10. Index và slice

Quy tắc lấy Index trong ma trận

- Giá trị trước dấu ',' là số dòng
- Giá trị sau dấu ',' là số cột
- Muốn chọn một dòng hoặc cột cụ thể thì cần thêm dấu ':'

```
#slice
a = np.array([(1, 2, 3), (4, 5, 6)])
print(a)
print('first row:', a[0])
print('second row:', a[1])
#indexing
print('Second column:', a[:,1])
print('first column', a[:, 0])
print('First two values of second row:', a[1, :2])
```

```
[[1 2 3]
 [4 5 6]]
first row: [1 2 3]
second row: [4 5 6]
Second column: [2 5]
first column [1 4]
First two values of second row: [4 5]
```


4. BÀI TẬP

1. Tạo mảng 20 số nguyên ngẫu nhiên kích thước 3×3 có giá trị trong khoảng $[0, 20]$. Viết chương trình kiểm tra tất cả các phần tử trong mảng khác 0
2. Tạo mảng 20 số nguyên ngẫu nhiên kích thước 3×3 có giá trị trong khoảng $[0, 20]$. Viết chương trình kiểm tra tồn tại một phần tử trong mảng khác 0
3. Sinh viên tìm hiểu và viết chương trình sử dụng các hàm sau `numpy.greater`, `numpy.greater_equal`, `numpy.less`, và `numpy.less_equal` để so sánh 2 mảng 1 chiều, 2 chiều.
4. Viết chương trình tạo mảng gồm 10 số 1, 10 số 0 và 10 số 5
5. Viết chương trình tạo mảng số nguyên chẵn từ $[30, 70]$
6. Viết chương trình tạo ma trận đơn vị 3×3 (đường chéo chính bằng 1, các phần tử còn lại 0)
7. Viết chương trình tạo mảng 10 phần tử trong khoảng $[15, 55]$. In các giá trị ngoài trừ phần tử đầu và cuối
8. Viết chương trình tạo mảng 20 phần tử trong khoảng $[0, 20]$ và đổi dấu những số nằm trong khoảng $[9, 15]$
9. Viết chương trình tạo ma trận 3×4 và mỗi phần tử có giá trị $[10, 21]$
10. Viết chương trình tạo ma trận 10×10 , các phần tử ở biên có giá trị 1, các phần tử bên trong có giá trị 0
11. Viết chương trình tạo ma trận 5×5 , các phần tử ở đường chéo chính là 1, 2, 3, 4, 5.
12. Viết chương trình tạo mảng $3 \times 3 \times 3$ và tính tổng theo mỗi dòng, mỗi cột.
13. Tạo 2 vector ngẫu nhiên 10 phần tử. Thực hiện inner product (`np.dot`)
14. Tạo ma trận $A = 4 \times 3$ và vector y gồm 3 phần tử ngẫu nhiên. Thêm vector y vào từng dòng của ma trận A

5. XÂY DỰNG ỨNG DỤNG

5.1. Tìm hiểu Tkinter/Flask/Streamlit

5.2. Ứng dụng các bài tập của phần 4 trên Tkinter/Flask/Streamlit